Xử Lý Tín Hiệu Số
HK261 – CO2036

BÀI THỰC HÀNH 3 XỬ LÝ TÍN HIỆU SỐ

Trần Minh Quang – 2412855

12/10/2026

Tóm tắt nội dung

Tổng Quan: Bài thực hành 3 xây dựng sáu hàm Scilab thao tác trên tín hiệu rời rạc hữu hạn: làm trễ (`delay`), làm sớm (`advance`), gập (`fold`), cộng (`add`), nhân (`multi`) và tích chập (`convolution`). Mỗi hàm được tính tay trước, sau đó chạy thử và vẽ đồ thị trên Scilab; kết quả mô phỏng của cả sáu bài đều khớp với tính toán và với ví dụ trong đề.

1 Giới Thiệu

Ở các bài trước, mỗi phép biến đổi tín hiệu được viết trực tiếp cho một tín hiệu cụ thể. Bài thực hành này đóng gói các phép toán thành hàm tự định nghĩa (`function ... endfunction`) để tái sử dụng cho tín hiệu bất kỳ. Do một mảng số không tự biết mẫu nào ứng với $n = 0$, mỗi tín hiệu đi kèm chỉ số gốc `xorigin`, và nhiệm vụ chính là tính đúng cả vector ngõ ra `yn` lẫn gốc mới `yorigin`. Công cụ sử dụng là Scilab 2026.1.0, vẽ tín hiệu rời rạc bằng `plot2d3` và `subplot`.

2 Mục Tiêu

- **Về lý thuyết:** ôn lại các phép toán cơ bản trên tín hiệu rời rạc: dịch thời gian (trễ/sớm), gập tín hiệu, cộng và nhân từng mẫu, tích chập tuyến tính giữa tín hiệu vào và đáp ứng xung của hệ LTI.
- **Về kỹ năng:** viết hàm script trong Scilab có nhiều tham số vào/ra; xử lý chỉ số mảng và đồng bộ trục thời gian giữa các tín hiệu có gốc khác nhau.
- **Về trực quan hóa:** vẽ tín hiệu vào và ra trên cùng một figure, cùng thang đo để dễ so sánh, và xuất kết quả ra console làm minh chứng.

3 Phương Pháp

3.1 Quy ước biểu diễn tín hiệu

Tín hiệu $x(n)$ được biểu diễn bởi vector $\mathbf{x} = [x_1, x_2, \ldots, x_L]$ và chỉ số gốc p (`xorigin`). Phần tử thứ i của vector ứng với thời điểm

$$n = i - p, \qquad i = 1, 2, \ldots, L, \tag{1}$$

nên tín hiệu xác định trên đoạn $n \in [\,1-p,\ L-p\,]$. Ví dụ $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$ có $L = 4, p = 3$, tức n chạy từ -2 đến 1. Trong code, trục thời gian được tạo bằng câu lệnh `nx = (1:length(xn)) - xorigin`, đúng theo công thức (1).

Ngược lại, nếu biết chỉ số thời gian bắt đầu n_{start} của một tín hiệu thì gốc của nó là

$$p = 1 - n_{\text{start}}. \tag{2}$$

3.2 Quy trình thực hiện

Với mỗi bài tập, em làm theo các bước:

1. **Tính toán giải tích:** từ định nghĩa phép toán, suy ra công thức cho `yn` và `yorigin`, sau đó tính tay kết quả cho ví dụ.
2. **Xây dựng hàm:** hiện thực công thức trong một hàm Scilab đúng chữ ký đề bài yêu cầu. Các hàm có tham số k đều kiểm tra điều kiện $k > 0$ ở đầu hàm.
3. **Trực quan hóa:** dùng `subplot` để vẽ các tín hiệu trên cùng một figure, đặt chung giới hạn trục (`data_bounds`) để các đồ thị cùng thang đo, đặt trục hoành đi qua gốc (`x_location = "origin"`).
4. **Kiểm chứng:** chạy script bằng `exec`, in `yn`, `yorigin` ra console và đối chiếu với kết quả tính tay.

4 Nội Dung Thực Hành

4.1 Bài tập 1 – Phép làm trễ

Bài tập 1. **function** [yn, yorigin] = **delay**(xn, xorigin, k) biểu diễn hàm trễ $y(n) = x(n-k)$, với $k > 0$, và

- Tín hiệu rời rạc $x(n)$ được biểu diễn bằng vector xn
- xorigin là vị trí gốc tọa độ của tín hiệu $x(n)$

- yn là vector của tín hiệu đầu ra
- yorigin là vị trí gốc tọa độ của tín hiệu $y(n)$
- Cả $x(n)$ và $y(n)$ đều được trình bày cùng một hình đồ thị

Ví dụ minh họa:

Để xác định tín hiệu trễ $y(n) = x(n-1)$ với tín hiệu ban đầu $x(n) = \{1,\ -2,\ \underset{\uparrow}{3},\ 6\}$, ta gọi hàm tự định nghĩa:

```
[yn, yorigin] = delay([1, -2, 3, 6], 3, 1);
```

Kết quả ngõ ra thu được:

- yn $= [1,\ -2,\ 3,\ 6]$
- yorigin $= 2$

Ngoài ra, hai tín hiệu $x(n)$ và $y(n)$ được vẽ trực quan hóa trên cùng một đồ thị để so sánh.

1. Cơ sở lý thuyết và tính toán:

Phép trễ $y(n) = x(n-k)$ với $k > 0$ dịch toàn bộ tín hiệu sang *phải* k đơn vị trên trục thời gian: mẫu $x(n_0)$ sẽ xuất hiện ở thời điểm $n_0 + k$ của $y(n)$. Giá trị các mẫu và thứ tự của chúng không đổi, nên vector ngõ ra giữ nguyên, chỉ có gốc tọa độ thay đổi.

Gọi p là gốc của $x(n)$. Phần tử thứ i ứng với $n = i - p$ trong $x(n)$, sau khi trễ nó ứng với $n = i - p + k = i - (p - k)$. So sánh với (1), gốc mới của $y(n)$ là:

$$\text{yn} = \text{xn}, \qquad \text{yorigin} = \text{xorigin} - k. \tag{3}$$

Với ví dụ $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$, $k = 1$: yorigin $= 3 - 1 = 2$, tức

$$y(n) = x(n-1) = \{1,\ \underset{\uparrow}{-2},\ 3,\ 6\}, \qquad n \in [-1,\ 2].$$

Kiểm tra: $y(0) = x(-1) = -2$ và $y(1) = x(0) = 3$, đúng với kết quả trên.

2. Mã nguồn thực thi trên Scilab:

```
function [yn, yorigin]=delay(xn, xorigin, k)
    // check k
    if k <= 0 then error("k phai > 0"); end

    // doi goc toa do (phep lam tre dich sang trai k vi tri trong mang)
    yn = xn;
    yorigin = xorigin - k;

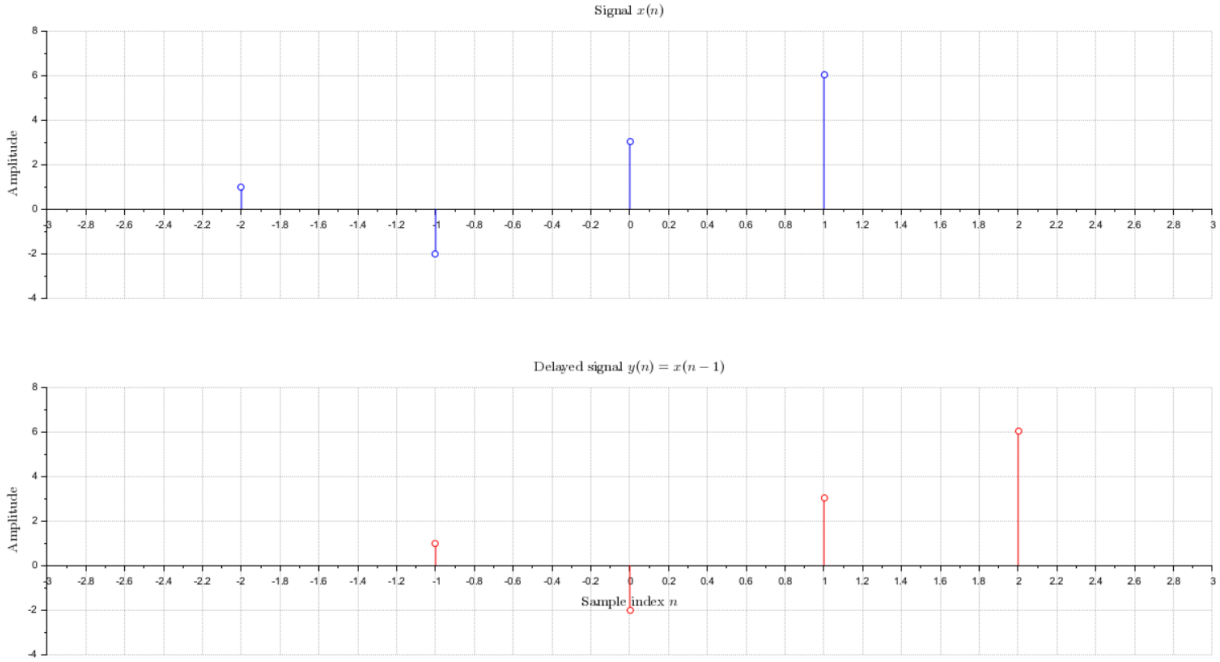
    // Truc thoi gian cho x(n) va y(n)
    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;
```

```
12
13      // gioi han hien thi truc hoanh va truc tung chung
14      n_min = min(min(nx), min(ny)) - 1;
15      n_max = max(max(nx), max(ny)) + 1;
16      y_min = min([xn, yn, 0]) - 1;
17      y_max = max([xn, yn, 0]) + 1;
18
19      clf();
20
21      // x(n)
22      subplot(2, 1, 1);
23      ax1 = gca();
24      ax1.x_location = "origin";
25      ax1.box = "off";
26      ax1.data_bounds = [n_min, y_min; n_max, y_max];
27      plot2d3(nx, xn, 2);
28      plot(nx, xn, 'bo');
29      xgrid(color('gray'));
30      title("$\text{Signal } x(n)$", "fontsize", 3);
31      ylabel("$\text{Amplitude}$", "fontsize", 3);
32
33      // y(n) = x(n - k)
34      subplot(2, 1, 2);
35      ax2 = gca();
36      ax2.x_location = "origin";
37      ax2.box = "off";
38      ax2.data_bounds = [n_min, y_min; n_max, y_max];
39      plot2d3(ny, yn, 5);
40      plot(ny, yn, 'ro');
41      xgrid(color('gray'));
42      title("$\text{Delayed signal } y(n) = x(n - " + string(k) + ")$", "
    ↪ fontsize", 3);
43      xlabel("$\text{Sample index } n$", "fontsize", 3);
44      ylabel("$\text{Amplitude}$", "fontsize", 3);
45  endfunction
46
47  xn = [1, -2, 3, 6];
48  xorigin = 3;
49  k = 1;
50
51  [yn, yorigin] = delay(xn, xorigin, k);
52  disp(yn, "yn =");
53  disp(yorigin, "yorigin =");
```

3. Kết quả mô phỏng và nhận xét:

```
Scilab 2026.1.0 Console
--> exec('D:\scilab-2026.1.0\scilab_workspace\lab3\Ex_3.1.sci', -1)
   1.  -2.   3.   6.
 "yn ="
  2.
 "yorigin ="
```

Hình 1: Kết quả in ra console của Bài tập 1



Hình 2: Đồ thị tín hiệu gốc $x(n)$ và tín hiệu trễ $y(n) = x(n-1)$

Console trả về yn $= [1, -2, 3, 6]$ và yorigin $= 2$, trùng với ví dụ của đề bài. Trên Hình 2, $x(n)$ chiếm đoạn $n \in [-2, 1]$ còn $y(n)$ chiếm đoạn $n \in [-1, 2]$: toàn bộ tín hiệu dịch sang phải đúng 1 mẫu, biên độ các mẫu không đổi. Mẫu tại gốc của $y(n)$ là -2, đúng như tính toán.

4.2 Bài tập 2 – Phép làm sớm

Bài tập 2. **function** [yn, yorigin] = **advance**(xn, xorigin, k) biểu diễn hàm tín hiệu sớm $y(n) = x(n + k)$, với $k > 0$, và

- Tín hiệu rời rạc $x(n)$ được biểu diễn bằng vector xn
- xorigin là vị trí gốc tọa độ của tín hiệu $x(n)$
- yn là vector của tín hiệu đầu ra
- yorigin là vị trí gốc tọa độ của tín hiệu $y(n)$

- Cả $x(n)$ và $y(n)$ đều được trình bày cùng một hình đồ thị

1. Cơ sở lý thuyết và tính toán:

Phép sớm $y(n) = x(n+k)$ với $k > 0$ ngược với phép trễ: tín hiệu dịch sang *trái* k đơn vị, mẫu $x(n_0)$ xuất hiện tại thời điểm $n_0 - k$. Lập luận tương tự Bài tập 1, phần tử thứ i ứng với $n = i - p - k = i - (p+k)$, do đó:

$$\text{yn} = \text{xn}, \qquad \text{yorigin} = \text{xorigin} + k. \tag{4}$$

Đề bài không cho ví dụ nên em dùng lại tín hiệu $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$ với $k = 1$: yorigin $= 3 + 1 = 4$, tức

$$y(n) = x(n+1) = \{1,\ -2,\ 3,\ \underset{\uparrow}{6}\}, \qquad n \in [-3,\ 0].$$

Kiểm tra: $y(0) = x(1) = 6$ và $y(-1) = x(0) = 3$.

2. Mã nguồn thực thi trên Scilab:

```
function [yn, yorigin]=advance(xn, xorigin, k)
    // check k
    if k <= 0 then error("k phai > 0"); end

    // doi goc toa do (phep lam som dich sang phai k vi tri trong mang)
    yn = xn;
    yorigin = xorigin + k;

    // Truc thoi gian cho x(n) va y(n)
    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;

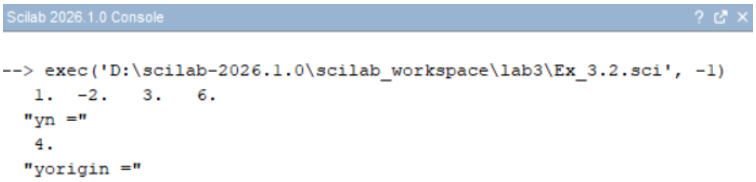
    // gioi han hien thi truc hoanh va truc tung chung
    n_min = min(min(nx), min(ny)) - 1;
    n_max = max(max(nx), max(ny)) + 1;
    y_min = min([xn, yn, 0]) - 1;
    y_max = max([xn, yn, 0]) + 1;

    clf();

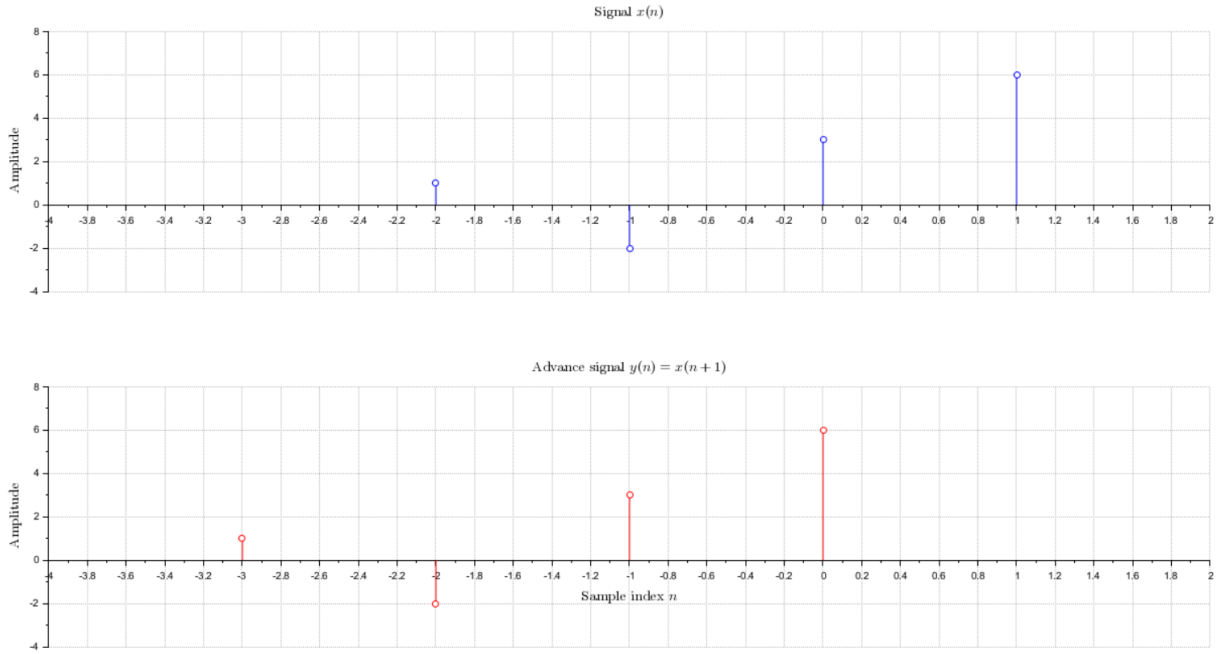
    // x(n)
    subplot(2, 1, 1);
    ax1 = gca();
    ax1.x_location = "origin";
    ax1.box = "off";
    ax1.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(nx, xn, 2);
    plot(nx, xn, 'bo');
    xgrid(color('gray'));
    title("$\text{Signal } x(n)$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);
```

```
32
33      // y(n) = x(n + k)
34      subplot(2, 1, 2);
35      ax2 = gca();
36      ax2.x_location = "origin";
37      ax2.box = "off";
38      ax2.data_bounds = [n_min, y_min; n_max, y_max];
39      plot2d3(ny, yn, 5);
40      plot(ny, yn, 'ro');
41      xgrid(color('gray'));
42      title("$\text{Advance signal } y(n) = x(n + " + string(k) + ")$", "
    ↪ fontsize", 3);
43      xlabel("$\text{Sample index } n$", "fontsize", 3);
44      ylabel("$\text{Amplitude}$", "fontsize", 3);
45  endfunction
46
47  xn = [1, -2, 3, 6];
48  xorigin = 3;
49  k = 1;
50
51  [yn, yorigin] = advance(xn, xorigin, k);
52  disp(yn, "yn =");
53  disp(yorigin, "yorigin =");
```

3. Kết quả mô phỏng và nhận xét:



Hình 3: Kết quả in ra console của Bài tập 2



Hình 4: Đồ thị tín hiệu gốc $x(n)$ và tín hiệu sớm $y(n) = x(n+1)$

Console trả về yn $= [1, -2, 3, 6]$ và yorigin $= 4$, khớp với tính toán. Trên Hình 4, $y(n)$ nằm trên đoạn $n \in [-3, 0]$, lệch sang trái 1 mẫu so với $x(n)$. Vì $k = 1$ nên gốc của $y(n)$ rơi đúng vào phần tử cuối cùng của vector (mẫu có giá trị 6).

4.3 Bài tập 3 – Phép gập tín hiệu

Bài tập 3. **function** [yn, yorigin] = **fold**(xn, xorigin) biểu diễn hàm lật $y(n) = x(-n)$, và

- Tín hiệu rời rạc $x(n)$ được biểu diễn bằng vector xn
- xorigin là vị trí gốc tọa độ của tín hiệu $x(n)$
- yn là vector của tín hiệu đầu ra
- yorigin là vị trí gốc tọa độ của tín hiệu $y(n)$
- Cả $x(n)$ và $y(n)$ đều được trình bày cùng một hình đồ thị

1. Cơ sở lý thuyết và tính toán:

Phép gập $y(n) = x(-n)$ lấy đối xứng tín hiệu qua trục $n = 0$: mẫu ở thời điểm n_0 chuyển sang $-n_0$. Khác với phép dịch, lần này *thứ tự* các mẫu bị đảo ngược, nên vector ngõ ra là vector ngõ vào viết ngược: yn$(j) =$ xn$(L - j + 1)$.

Mẫu gốc $x(0)$ nằm ở phần tử thứ p của `xn`. Sau khi đảo, nó nằm ở vị trí $L - p + 1$, và vì $y(0) = x(0)$ nên:

$$\text{yn} = [x_L, x_{L-1}, \ldots, x_1], \qquad \text{yorigin} = L - \text{xorigin} + 1. \tag{5}$$

Với $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$ ($L = 4, p = 3$): yorigin $= 4 - 3 + 1 = 2$, tức

$$y(n) = x(-n) = \{6, \underset{\uparrow}{3}, -2, 1\}, \qquad n \in [-1, 2].$$

Kiểm tra: $y(-1) = x(1) = 6$ và $y(2) = x(-2) = 1$.

2. Mã nguồn thực thi trên Scilab:

```
function [yn, yorigin]=fold(xn, xorigin)
    // Dao nguoc thu tu mang va cap nhat goc toa do moi
    yn = xn($:-1:1);
    yorigin = length(xn) - xorigin + 1;

    // Truc thoi gian cho x(n) va y(n)
    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;

    // gioi han hien thi truc hoanh va truc tung chung
    n_min = min(min(nx), min(ny)) - 1;
    n_max = max(max(nx), max(ny)) + 1;
    y_min = min([xn, yn, 0]) - 1;
    y_max = max([xn, yn, 0]) + 1;

    clf();

    // x(n)
    subplot(2, 1, 1);
    ax1 = gca();
    ax1.x_location = "origin";
    ax1.box = "off";
    ax1.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(nx, xn, 2);
    plot(nx, xn, 'bo');
    xgrid(color('gray'));
    title("$\text{Signal } x(n)$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);

    // y(n) = x(-n)
    subplot(2, 1, 2);
    ax2 = gca();
    ax2.x_location = "origin";
    ax2.box = "off";
    ax2.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(ny, yn, 5);
    plot(ny, yn, 'ro');
    xgrid(color('gray'));
    title("$\text{Folded signal } y(n) = x(-n)$", "fontsize", 3);
    xlabel("$\text{Sample index } n$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);
```

```
endfunction

xn = [1, -2, 3, 6];
xorigin = 3;

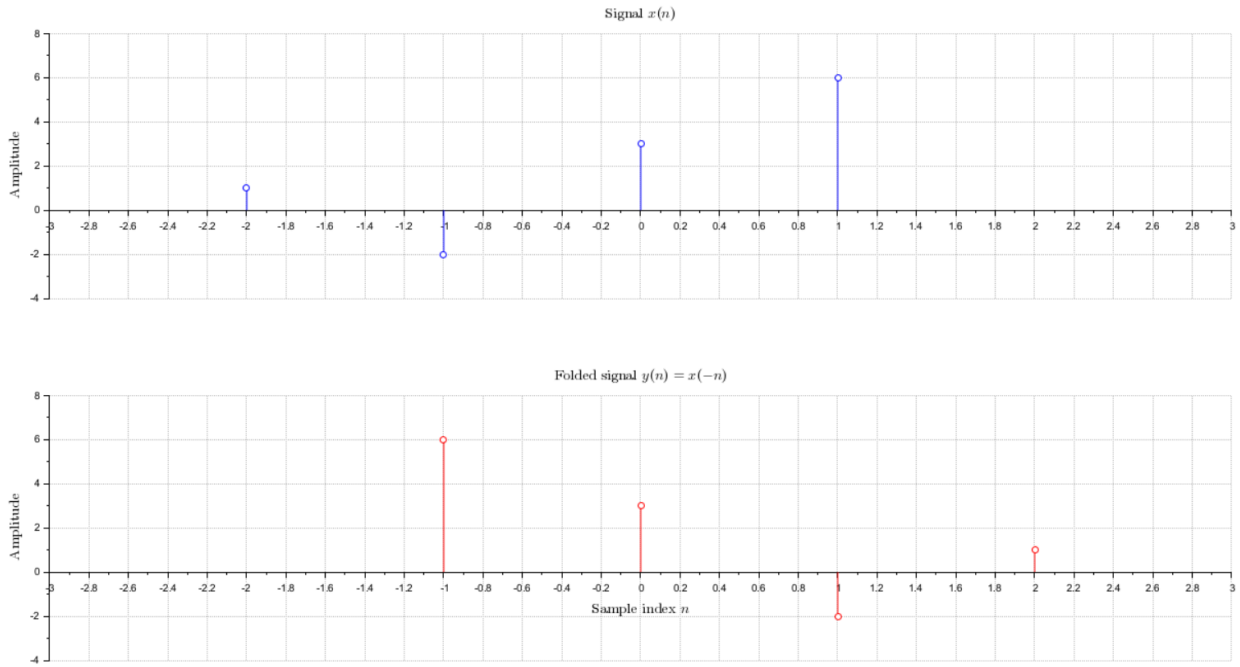
[yn, yorigin] = fold(xn, xorigin);
disp(yn, "yn =");
disp(yorigin, "yorigin =");
```

3. Kết quả mô phỏng và nhận xét:

```
Scilab 2026.1.0 Console

--> exec('D:\scilab-2026.1.0\scilab_workspace\lab3\Ex_3.3.sci', -1)
   6.   3.  -2.   1.
 "yn ="
  2.
 "yorigin ="
```

Hình 5: Kết quả in ra console của Bài tập 3



Hình 6: Đồ thị tín hiệu gốc $x(n)$ và tín hiệu gập $y(n) = x(-n)$

Console trả về yn $= [6, 3, -2, 1]$ và yorigin $= 2$, đúng như tính tay. Hình 6 cho thấy $y(n)$ là ảnh đối xứng của $x(n)$ qua trục tung: mẫu tại $n = 0$ (giá trị 3) đứng yên, các mẫu còn lại đổi phía. Chẳng hạn mẫu -2 ở $n = -1$ chuyển sang $n = 1$, mẫu 6 ở $n = 1$ chuyển sang $n = -1$.

4.4 Bài tập 4 – Phép cộng hai tín hiệu

Bài tập 4. **function** [yn, yorigin] = **add**(x1n, x1origin, x2n, x2origin) biểu diễn hàm tổng $y(n) = x_1(n) + x_2(n)$, và

- Tín hiệu rời rạc $x_1(n), x_2(n)$ được biểu diễn bằng vector x1n,x2n
- x1origin và x2origin là vị trí gốc tọa độ của tín hiệu $x_1(n), x_2(n)$
- yn là vector của tín hiệu đầu ra
- yorigin là vị trí gốc tọa độ của tín hiệu $y(n)$
- Cả $x_1(n), x_2(n)$ và $y(n)$ đều được trình bày cùng một hình đồ thị

Ví dụ minh họa:

Để xác định $y(n) = x_1(n) + x_2(n)$ với $x_1(n) = \{\underset{\uparrow}{0}, 1, 3, -2\}$ và $x_2(n) = \{1, \underset{\uparrow}{1}, 2, 3\}$, ta gọi:

```
[yn, yorigin] = add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2);
```

Kết quả ngõ ra: yn $= [1, 1, 3, 6, -2]$ và yorigin $= 2$.

1. Cơ sở lý thuyết và tính toán:

Phép cộng hai tín hiệu được thực hiện *theo từng mẫu tại cùng thời điểm*: $y(n) = x_1(n) + x_2(n)$. Vì hai vector có thể có gốc và chiều dài khác nhau, ta không thể cộng trực tiếp hai mảng mà phải đồng bộ trục thời gian trước:

1. Tính trục thời gian n_1, n_2 của từng tín hiệu theo (1).
2. Lấy miền chung $n \in [n_{\text{start}}, n_{\text{end}}]$ với $n_{\text{start}} = \min(n_1, n_2)$, $n_{\text{end}} = \max(n_1, n_2)$.
3. Bù 0 cho mỗi tín hiệu tại các thời điểm nó không xác định (vì tín hiệu hữu hạn bằng 0 ngoài miền của nó), rồi cộng từng phần tử.
4. Gốc mới tính theo (2): yorigin $= 1 - n_{\text{start}}$.

Với ví dụ của đề: $x_1(n)$ xác định trên $n \in [0, 3]$, $x_2(n)$ trên $n \in [-1, 2]$, nên miền chung là $n \in [-1, 3]$:

n	-1	0	1	2	3
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	1	1	2	3	0
$y(n) = x_1(n) + x_2(n)$	1	1	3	6	-2

Bảng 1: Tính tay phép cộng $y(n) = x_1(n) + x_2(n)$

Vậy $y(n) = \{1, \underset{\uparrow}{1}, 3, 6, -2\}$ và yorigin $= 1 - (-1) = 2$.

2. Mã nguồn thực thi trên Scilab:

```
function [yn, yorigin]=add(x1n, x1origin, x2n, x2origin)
    // Truc thoi gian cho x1(n) va x2(n)
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;

    // Mien xac dinh chung n cho phep cong
    n_start = min(min(n1), min(n2));
    n_end = max(max(n1), max(n2));
    ny = n_start:n_end;

    // Dong bo chieu dai va bu 0 cho hai tin hieu
    x1_aligned = zeros(1, length(ny));
    x2_aligned = zeros(1, length(ny));

    // Dien gia tri vao dung vi tri chi so thoi gian
    idx1 = (n1 - n_start + 1);
    idx2 = (n2 - n_start + 1);
    x1_aligned(idx1) = x1n;
    x2_aligned(idx2) = x2n;

    // Tin hieu tong va goc toa do moi
    yn = x1_aligned + x2_aligned;
    yorigin = 1 - n_start;

    // gioi han hien thi truc hoanh va truc tung
    n_min = n_start - 1;
    n_max = n_end + 1;
    y_min = min([x1_aligned, x2_aligned, yn, 0]) - 1;
    y_max = max([x1_aligned, x2_aligned, yn, 0]) + 1;

    clf();

    // x1(n)
    subplot(3, 1, 1);
    ax1 = gca();
    ax1.x_location = "origin";
    ax1.box = "off";
    ax1.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(ny, x1_aligned, 2);
    plot(ny, x1_aligned, 'bo');
    xgrid(color('gray'));
    title("$\text{Signal } x_1(n)$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);

    // x2(n)
    subplot(3, 1, 2);
    ax2 = gca();
```

```
    ax2.x_location = "origin";
    ax2.box = "off";
    ax2.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(ny, x2_aligned, 2);
    plot(ny, x2_aligned, 'bo');
    xgrid(color('gray'));
    title("$\text{Signal } x_2(n)$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);

    // y(n) = x1(n) + x2(n)
    subplot(3, 1, 3);
    ax3 = gca();
    ax3.x_location = "origin";
    ax3.box = "off";
    ax3.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(ny, yn, 5);
    plot(ny, yn, 'ro');
    xgrid(color('gray'));
    title("$\text{Output signal } y(n) = x_1(n) + x_2(n)$", "fontsize", 3);
    xlabel("$\text{Sample index } n$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);
endfunction

x1n = [0, 1, 3, -2];
x1origin = 1;
x2n = [1, 1, 2, 3];
x2origin = 2;

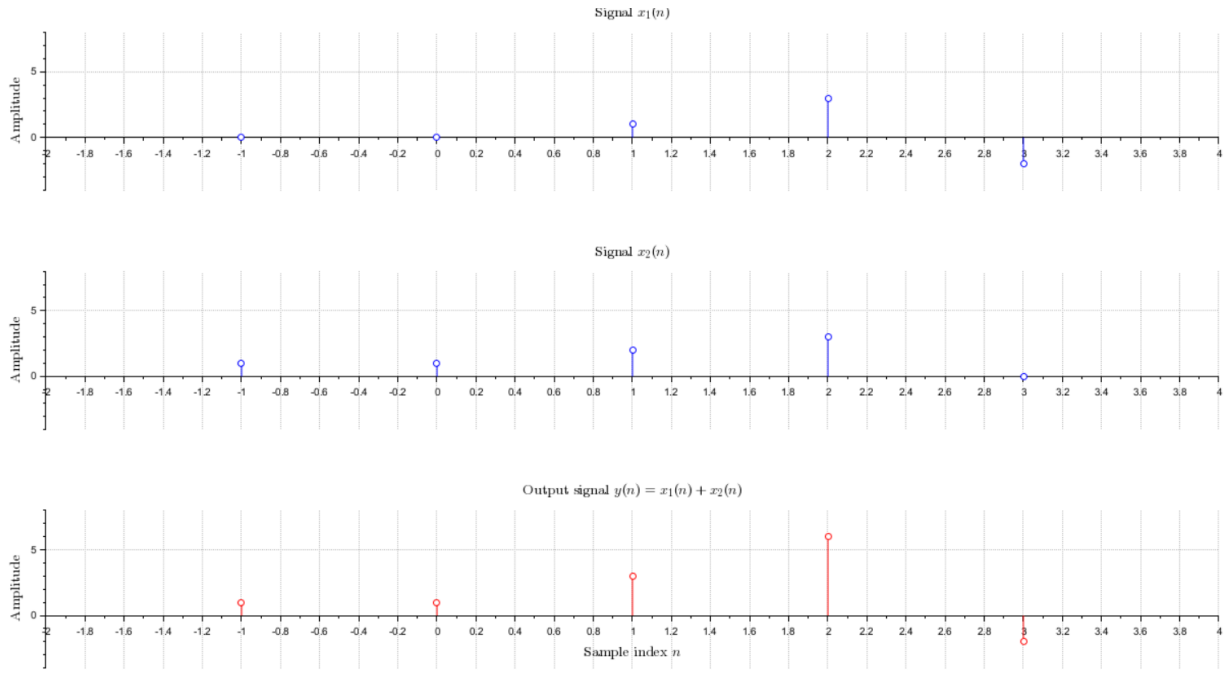
[yn, yorigin] = add(x1n, x1origin, x2n, x2origin);
disp(yn, "yn =");
disp(yorigin, "yorigin =");
```

3. Kết quả mô phỏng và nhận xét:

```
Scilab 2026.1.0 Console

--> exec('D:\scilab-2026.1.0\scilab_workspace\lab3\Ex_3.4.sci', -1)
   1.   1.   3.   6.  -2.
  "yn ="
   2.
  "yorigin ="
```

Hình 7: Kết quả in ra console của Bài tập 4



Hình 8: Đồ thị hai tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tổng $y(n)$

Console trả về yn $= [1, 1, 3, 6, -2]$ và yorigin $= 2$, đúng với ví dụ của đề bài. Trên Hình 8, cả ba tín hiệu được vẽ trên cùng miền $n \in [-1, 3]$. Có thể thấy rõ các mẫu được bù 0: $x_1(n)$ tại $n = -1$ và $x_2(n)$ tại $n = 3$. Mỗi mẫu của $y(n)$ đúng bằng tổng hai mẫu nằm thẳng cột phía trên, ví dụ tại $n = 2$: $3 + 3 = 6$.

Lúc đầu em tính gốc mới bằng `find(ny == 0)`. Cách này đúng với ví dụ trên nhưng sẽ trả về rỗng nếu miền của $y(n)$ không chứa $n = 0$, chẳng hạn khi cộng hai tín hiệu đã bị làm trễ nhiều mẫu. Vì vậy em chuyển sang công thức `yorigin = 1 - n_start`, luôn đúng trong mọi trường hợp.

4.5 Bài tập 5 – Phép nhân hai tín hiệu

Bài tập 5. **function** [yn, yorigin] = **multi**(x1n, x1origin, x2n, x2origin) biểu diễn hàm nhân $y(n) = x_1(n) \cdot x_2(n)$, và

- Tín hiệu rời rạc $x_1(n), x_2(n)$ được biểu diễn bằng vector x1n,x2n
- x1origin và x2origin là vị trí gốc tọa độ của tín hiệu $x_1(n), x_2(n)$
- yn là vector của tín hiệu đầu ra
- yorigin là vị trí gốc tọa độ của tín hiệu $y(n)$
- Cả $x_1(n), x_2(n)$ và $y(n)$ đều được trình bày cùng một hình đồ thị

1. Cơ sở lý thuyết và tính toán:

Phép nhân cũng được thực hiện theo từng mẫu: $y(n) = x_1(n) \cdot x_2(n)$. Quy trình đồng bộ trục thời gian giống hệt Bài tập 4; điểm khác duy nhất là thay phép cộng bằng phép nhân từng phần tử (toán tử `.*` trong Scilab). Một hệ quả đáng chú ý: tại những thời điểm chỉ một trong hai tín hiệu có giá trị, tín hiệu còn lại được bù 0 nên tích bằng 0. Vì vậy $y(n)$ chỉ có thể khác 0 trên *phần giao* của hai miền xác định.

Em dùng lại hai tín hiệu của Bài tập 4 để dễ so sánh:

n	-1	0	1	2	3
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	1	1	2	3	0
$y(n) = x_1(n) \cdot x_2(n)$	0	0	2	9	0

Bảng 2: Tính tay phép nhân $y(n) = x_1(n) \cdot x_2(n)$

Vậy $y(n) = \{0, \underset{\uparrow}{0}, 2, 9, 0\}$ và yorigin $= 1 - (-1) = 2$.

2. Mã nguồn thực thi trên Scilab:

```
function [yn, yorigin]=multi(x1n, x1origin, x2n, x2origin)
    // Truc thoi gian cho x1(n) va x2(n)
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;

    // Mien xac dinh chung n cho phep nhan
    n_start = min(min(n1), min(n2));
    n_end = max(max(n1), max(n2));
    ny = n_start:n_end;

    // Dong bo chieu dai va bu 0 cho hai tin hieu
    x1_aligned = zeros(1, length(ny));
    x2_aligned = zeros(1, length(ny));

    // Dien gia tri vao dung vi tri chi so thoi gian
    idx1 = (n1 - n_start + 1);
    idx2 = (n2 - n_start + 1);
    x1_aligned(idx1) = x1n;
    x2_aligned(idx2) = x2n;

    // Tin hieu tich (nhan tung mau) va goc toa do moi
    yn = x1_aligned .* x2_aligned;
    yorigin = 1 - n_start;

    // gioi han hien thi truc hoanh va truc tung
    n_min = n_start - 1;
    n_max = n_end + 1;
    y_min = min([x1_aligned, x2_aligned, yn, 0]) - 1;
    y_max = max([x1_aligned, x2_aligned, yn, 0]) + 1;
```

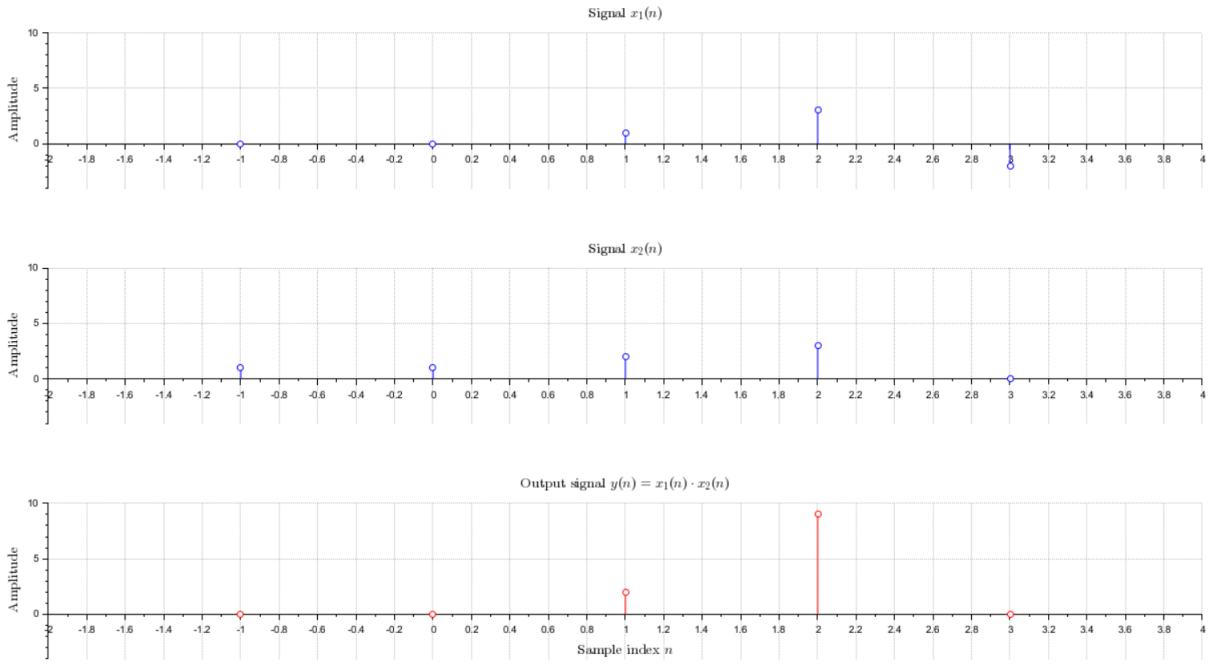
```
30
31      clf();
32
33      // x1(n)
34      subplot(3, 1, 1);
35      ax1 = gca();
36      ax1.x_location = "origin";
37      ax1.box = "off";
38      ax1.data_bounds = [n_min, y_min; n_max, y_max];
39      plot2d3(ny, x1_aligned, 2);
40      plot(ny, x1_aligned, 'bo');
41      xgrid(color('gray'));
42      title("$\text{Signal } x_1(n)$", "fontsize", 3);
43      ylabel("$\text{Amplitude}$", "fontsize", 3);
44
45      // x2(n)
46      subplot(3, 1, 2);
47      ax2 = gca();
48      ax2.x_location = "origin";
49      ax2.box = "off";
50      ax2.data_bounds = [n_min, y_min; n_max, y_max];
51      plot2d3(ny, x2_aligned, 2);
52      plot(ny, x2_aligned, 'bo');
53      xgrid(color('gray'));
54      title("$\text{Signal } x_2(n)$", "fontsize", 3);
55      ylabel("$\text{Amplitude}$", "fontsize", 3);
56
57      // y(n) = x1(n) . x2(n)
58      subplot(3, 1, 3);
59      ax3 = gca();
60      ax3.x_location = "origin";
61      ax3.box = "off";
62      ax3.data_bounds = [n_min, y_min; n_max, y_max];
63      plot2d3(ny, yn, 5);
64      plot(ny, yn, 'ro');
65      xgrid(color('gray'));
66      title("$\text{Output signal } y(n) = x_1(n) \cdot x_2(n)$", "fontsize",
    ↪  3);
67      xlabel("$\text{Sample index } n$", "fontsize", 3);
68      ylabel("$\text{Amplitude}$", "fontsize", 3);
69  endfunction
70
71  x1n = [0, 1, 3, -2];
72  x1origin = 1;
73  x2n = [1, 1, 2, 3];
74  x2origin = 2;
75
76  [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin);
```

```
77 disp(yn, "yn =");
78 disp(yorigin, "yorigin =");
```

3. Kết quả mô phỏng và nhận xét:

```
Scilab 2026.1.0 Console
--> exec('D:\scilab-2026.1.0\scilab_workspace\lab3\Ex_3.5.sci', -1)
   0.    0.    2.    9.    0.
 "yn ="
  2.
 "yorigin ="
```

Hình 9: Kết quả in ra console của Bài tập 5



Hình 10: Đồ thị hai tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tích $y(n)$

Console trả về yn $= [0, 0, 2, 9, 0]$ và yorigin $= 2$, khớp với bảng tính tay. Trên Hình 10, $y(n)$ chỉ khác 0 tại $n = 1$ và $n = 2$. Tại $n = -1$ và $n = 3$, một trong hai tín hiệu bằng 0 do bù thêm nên tích bằng 0. Tại $n = 0$ thì bản thân $x_1(0) = 0$. Điều này đúng với nhận xét lý thuyết ở trên.

4.6 Bài tập 6 – Tích chập

Bài tập 6. **function** [yn, yorigin] = **convolution**(xn, xorigin, hn, horigin) biểu diễn hàm tích chập $y(n) = x(n) * h(n)$, và

- $x(n)$ là tín hiệu ngõ vào và $h(n)$ là hàm đặc trưng (đáp ứng xung) của hệ thống.
- `xorigin` và `horigin` lần lượt biểu thị vị trí gốc tọa độ của $x(n)$ và $h(n)$.

- `yn` là vector biểu diễn tín hiệu ngõ ra $y(n)$.
- `yorigin` biểu thị vị trí gốc tọa độ của tín hiệu ngõ ra $y(n)$.

1. Cơ sở lý thuyết và tính toán:

Với hệ tuyến tính bất biến theo thời gian (LTI) có đáp ứng xung $h(n)$, ngõ ra ứng với ngõ vào $x(n)$ là tích chập tuyến tính:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)\, h(n-k). \tag{6}$$

Với hai tín hiệu hữu hạn có chiều dài L_x và L_h, mỗi cặp mẫu $x(k)$, $h(m)$ đóng góp tích $x(k)h(m)$ vào $y(k+m)$. Từ đó suy ra:

- Chiều dài ngõ ra: $L_y = L_x + L_h - 1$.
- Thời điểm bắt đầu: $n_{y,\text{start}} = n_{x,\text{start}} + n_{h,\text{start}} = (1 - p_x) + (1 - p_h)$.
- Theo (2), gốc mới: yorigin $= 1 - n_{y,\text{start}} = p_x + p_h - 1$.

Trong code, em hiện thực đúng ý tưởng "mỗi cặp mẫu cộng dồn vào một vị trí" bằng hai vòng lặp: phần tử thứ i của `xn` nhân với phần tử thứ j của `hn` được cộng vào phần tử thứ $i + j - 1$ của `yn`.

Ví dụ: $x(n) = \{1,\ \underset{\uparrow}{2},\ 3\}$ ($p_x = 2$) và $h(n) = \{\underset{\uparrow}{1},\ -1,\ 2\}$ ($p_h = 1$). Khi đó $L_y = 3+3-1 = 5$, yorigin $= 2 + 1 - 1 = 2$, và $y(n)$ xác định trên $n \in [-1, 3]$:

$$\begin{aligned}
y(-1) &= x(-1)h(0) = 1 \cdot 1 = 1 \\
y(0) &= x(-1)h(1) + x(0)h(0) = 1 \cdot (-1) + 2 \cdot 1 = 1 \\
y(1) &= x(-1)h(2) + x(0)h(1) + x(1)h(0) = 1 \cdot 2 + 2 \cdot (-1) + 3 \cdot 1 = 3 \\
y(2) &= x(0)h(2) + x(1)h(1) = 2 \cdot 2 + 3 \cdot (-1) = 1 \\
y(3) &= x(1)h(2) = 3 \cdot 2 = 6
\end{aligned}$$

Vậy $y(n) = \{1,\ \underset{\uparrow}{1},\ 3,\ 1,\ 6\}$.

2. Mã nguồn thực thi trên Scilab:

```
function [yn, yorigin]=convolution(xn, xorigin, hn, horigin)
    // Chieu dai cua x(n), h(n) va y(n)
    Lx = length(xn);
    Lh = length(hn);
    Ly = Lx + Lh - 1;

    // Tich chap: y(n) = sum_k x(k).h(n - k)
    yn = zeros(1, Ly);
    for i = 1:Lx
        for j = 1:Lh
            yn(i + j - 1) = yn(i + j - 1) + xn(i) * hn(j);
```

```
        end
    end

    // Goc toa do moi (chi so bat dau cua y = tong chi so bat dau cua x va
    ↪ h)
    yorigin = xorigin + horigin - 1;

    // Truc thoi gian cho x(n), h(n) va y(n)
    nx = (1:Lx) - xorigin;
    nh = (1:Lh) - horigin;
    ny = (1:Ly) - yorigin;

    // gioi han hien thi truc hoanh va truc tung chung
    n_min = min([nx, nh, ny]) - 1;
    n_max = max([nx, nh, ny]) + 1;
    y_min = min([xn, hn, yn, 0]) - 1;
    y_max = max([xn, hn, yn, 0]) + 1;

    clf();

    // x(n)
    subplot(3, 1, 1);
    ax1 = gca();
    ax1.x_location = "origin";
    ax1.box = "off";
    ax1.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(nx, xn, 2);
    plot(nx, xn, 'bo');
    xgrid(color('gray'));
    title("$\text{Input signal } x(n)$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);

    // h(n)
    subplot(3, 1, 2);
    ax2 = gca();
    ax2.x_location = "origin";
    ax2.box = "off";
    ax2.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(nh, hn, 2);
    plot(nh, hn, 'bo');
    xgrid(color('gray'));
    title("$\text{Impulse response } h(n)$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);

    // y(n) = x(n) * h(n)
    subplot(3, 1, 3);
    ax3 = gca();
    ax3.x_location = "origin";
```

```
    ax3.box = "off";
    ax3.data_bounds = [n_min, y_min; n_max, y_max];
    plot2d3(ny, yn, 5);
    plot(ny, yn, 'ro');
    xgrid(color('gray'));
    title("$\text{Output signal } y(n) = x(n) * h(n)$", "fontsize", 3);
    xlabel("$\text{Sample index } n$", "fontsize", 3);
    ylabel("$\text{Amplitude}$", "fontsize", 3);
endfunction

xn = [1, 2, 3];
xorigin = 2;
hn = [1, -1, 2];
horigin = 1;

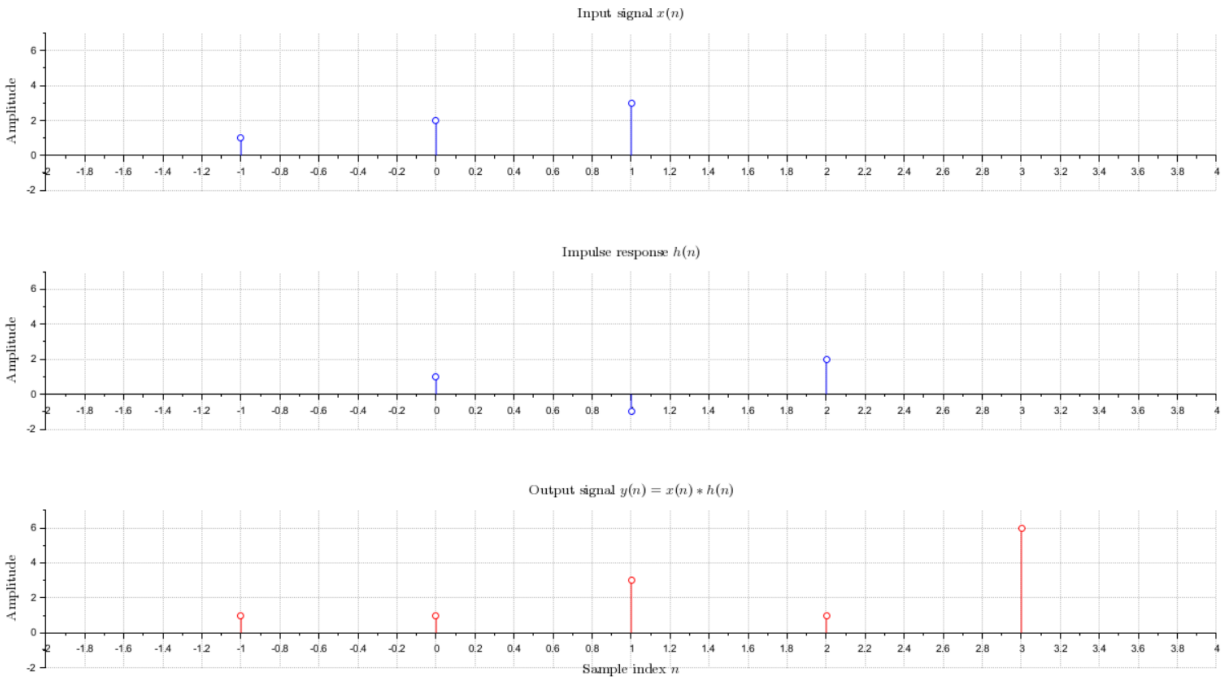
[yn, yorigin] = convolution(xn, xorigin, hn, horigin);
disp(yn, "yn =");
disp(yorigin, "yorigin =");
```

3. Kết quả mô phỏng và nhận xét:

```
Scilab 2026.1.0 Console

--> exec('D:\scilab-2026.1.0\scilab_workspace\lab3\Ex_3.6.sci', -1)
   1.    1.    3.    1.    6.
 "yn ="
   2.
 "yorigin ="
```

Hình 11: Kết quả in ra console của Bài tập 6



Hình 12: Đồ thị tín hiệu vào $x(n)$, đáp ứng xung $h(n)$ và tín hiệu ra $y(n) = x(n) * h(n)$

Console trả về yn $= [1, 1, 3, 1, 6]$ và yorigin $= 2$, trùng với kết quả tính tay. Đề bài không bắt buộc vẽ đồ thị cho bài này, nhưng em vẫn vẽ cả ba tín hiệu (Hình 12) để quan sát. Tín hiệu ra $y(n)$ dài 5 mẫu, bắt đầu tại $n = -1 = (-1) + 0$, tức tổng hai thời điểm bắt đầu của $x(n)$ và $h(n)$. Điều này khẳng định công thức chiều dài và gốc tọa độ ở trên. Ngoài ra, kết quả này cũng có thể đối chiếu nhanh với hàm có sẵn `conv(xn, hn)` của Scilab: hàm này cho cùng vector $[1, 1, 3, 1, 6]$ nhưng không trả về gốc tọa độ.

5 Bàn Luận

Kết quả của cả sáu hàm trên Scilab đều trùng với kết quả tính tay và với ví dụ trong đề, các đồ thị thể hiện đúng bản chất từng phép toán: trễ/sớm chỉ tịnh tiến tín hiệu, gập lấy đối xứng qua $n = 0$, cộng/nhân thực hiện theo từng mẫu trên miền chung, còn tích chập cho tín hiệu dài $L_x + L_h - 1$ mẫu. Qua bài thực hành, em đã hiểu rõ hơn cách các phép toán cơ bản trên tín hiệu rời rạc tác động lên cả giá trị lẫn trục thời gian của tín hiệu, đặc biệt phép tích chập là nên tảng cho các bài thực hành tiếp theo.

6 Tài Liệu Tham Khảo

- Đường link GitHub chứa mã nguồn thực hành source code SciLab: https://github.com/qanetta/DigitalSignalProcessing-Lab.git